

2026-07-03



Social Gaze Task

2026.06.26(금)

미팅내용

- Graph-confidence gated VLM 성능, 분석 공유 - **graph low-confidence**에서도 VLM이 성능 크게 올리지 못하거나 떨어뜨림.
- Graph confidence와 성능과의 관계 봤을때 graph confidence는 correctness의 지표 될 수 있으나, **fixed routing은 좋지 않음.**
- Graph Router 학습시킨 graph-vlm 성능 공유 - 특히 SA 성능 잘 안오름
- Glance for context논문 아이디어/설계 공유, 우리 프레임워크에 적용 가능해보이는것 공유(router, refiner..)

다음할일

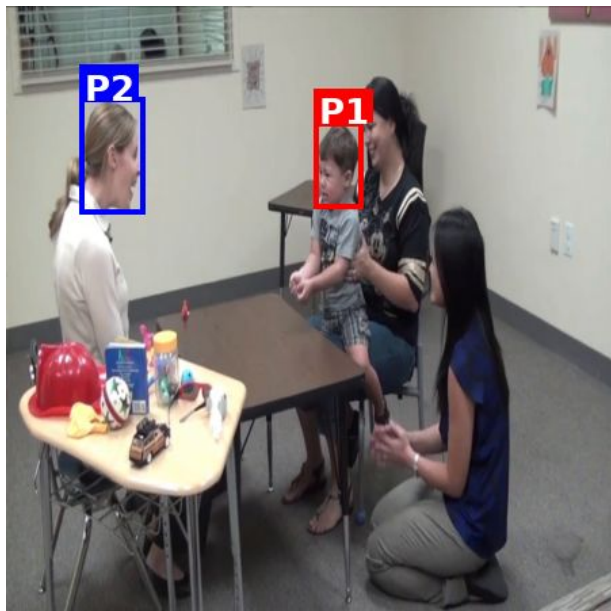
- 슬라이드에 vlm에 어떤 input들 들어가는지, output이 어떻게나오는지 이 과정 잘 드러나게 구성하기.
- graph feature text + VLM fine tune 성능 확인해보기
- graph 보다 성능 올릴 수 있는 방식 여러가지 실험해보기, 여러 요소 한번에 하지 말고 하나씩만 적용하면서.

VLM Fine-Tuning 실험 공통 세팅

- **Task**
: 사람쌍 (i,j)의 사회적 시선 — **LAH**(i가 j를 봄, directed), **LAEO**(상호응시= $\text{LAH}_{ij} \wedge \text{LAH}_{ji}$), **SA**(공동주시, triadic).
- **지표**
: **F1_LAH** / **F1_LAEO** (per-target-argmax + thr0.5), **AP_SA** (per-pair AP).
- **Graph(frozen V14.5) 제공 feature:**
per-person **v_src** (256, looker/emitter 임베딩) · **v_tgt** (256, target/receiver) · **gaze_vecs** · **gaze_point** · **null_in/out_logits**, **per-pair edge_pp** (256, directed) · **overlap** · **align** · **lah/laeo/sa_logits** · **edge_null_in/out** (256).
- **공통 VLM(A-E)**
: **Qwen3-VL-8B** + LoRA(r16 α 32), target=LM의 {q,k,v,o,gate,up,down}_proj, vision frozen.
학습 = 답 토큰(yes/no) CE, WeightedRandomSampler((task,answer) 균형), bs4-8, lr1e-4.

Epoch 3 순수 학습 1.8h + val 평가 3h 합쳐 약 5h 소요,
MIG 기준 각각 test 소요시간 11시간 30분, vram 약 40gb

실험별 세팅



A. vision-only

- 입력

: **pair overlay PNG**(원본 이미지 + 사람 i=빨강 박스, j=파랑 박스, Person id) + 쿼리별 텍스트 프롬프트 (system + "Is P1 looking at P2?" / "Are P1 and P2 looking at each other?" / "Are P1 and P2 looking at the same thing or person?" → Answer yes or no.)

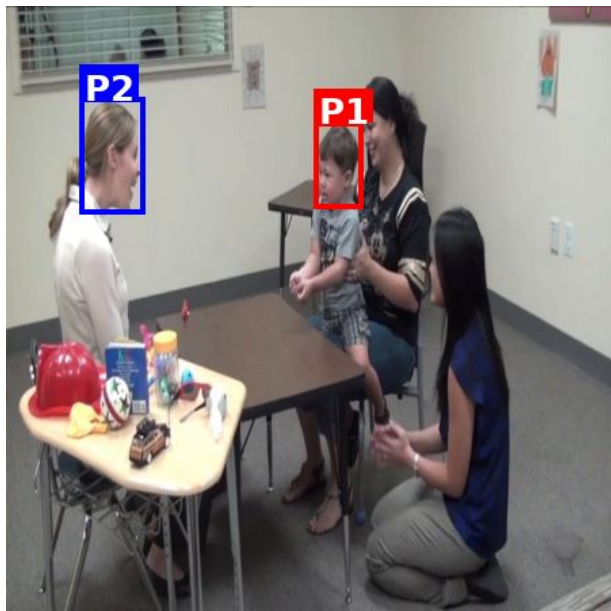
- 주입

: 없음. graph 정보 0. VLM은 overlay를 vision encoder로만 봄. -> graph와 완전 독립.

- 학습

: LoRA + 답토큰 CE.

실험별 세팅



Graph 판정 no
Vision-only finetuned VLM yes
Vision+Graph finetuned VLM yes

B. graph-text

• 입력

: 이전 세팅 + graph-text 블록을 텍스트로 prepend

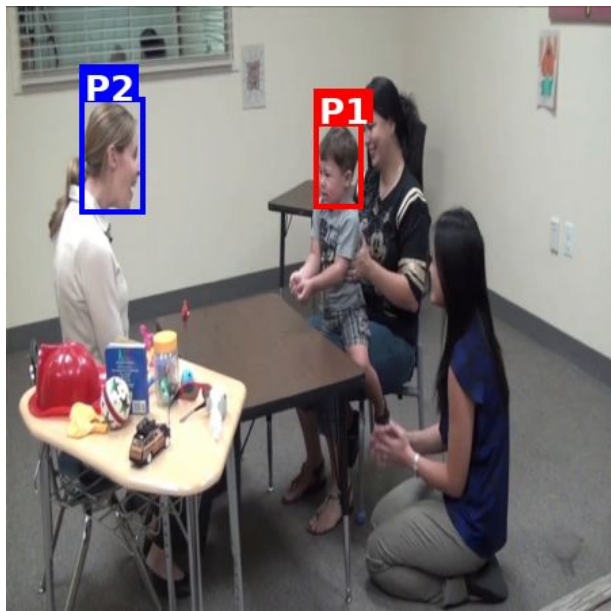
• graph-text (LAH의 경우)

- P1 (red) head box [0.51,0.19,0.59,0.33]; P2 (blue) head box [0.12,0.15,0.23,0.34].
- **P1 (red) gaze points left, near horizontal; predicted gaze point (0.20,0.26). Aim at P2 (blue) (cos of gaze vs P1머리중심->P2머리중심): +0.99 (+1=straight at P2, -1=opposite). Gaze-heatmap landing on P2 (blue)'s head: 0.57 (0=miss, 1=hit).**
- **Graph edge score P1->P2: 0.48;**
P1->nobody/out-of-frame: 0.04. Per-frame P1->P2 scores: 0.11, 0.12, 0.48, 0.56, 0.61. Other people predicted to look at P2: 0.

• 학습

: LoRA + 답토큰 CE.

실험별 세팅



Graph 판정 no
Vision-only finetuned VLM yes
Vision+Graph finetuned VLM yes

B. graph-text

• 입력

: 이전 세팅 + graph-text 블록을 텍스트로 prepend

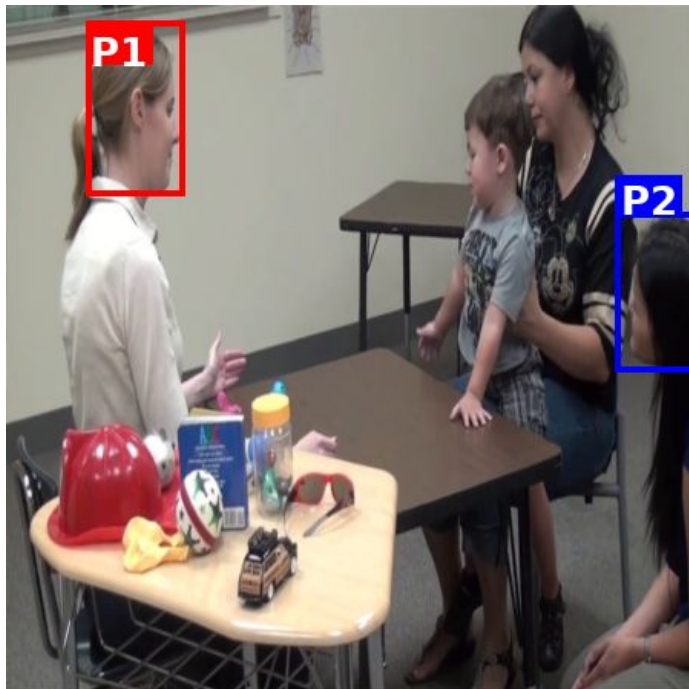
• graph-text (LAEO의 경우)

- P1 (red) head box [0.50,0.19,0.58,0.33]; P2 (blue) head box [0.13,0.15,0.23,0.34].
- **P1 (red) gaze points left, near horizontal (gaze point (0.21,0.26)); aim at P2: +1.00; gaze-heatmap landing on P2: 0.57.**
- **P2 (blue) gaze points right, ~13 deg below horizontal (gaze point (0.52,0.27)); aim at P1: +0.98; gaze-heatmap landing on P1: 0.32.**
- **Graph edge scores: P1->P2 0.47, P2->P1 0.97, mutual 0.34. Per-frame mutual scores: 0.35, 0.33, 0.34, 0.04, 0.06.**

• 학습

: LoRA + 답토큰 CE.

실험별 세팅



Graph 판정 0.38

Vision-only finetuned VLM no

Vision+Graph finetuned VLM no

B. graph-text

• 입력

: 이전 세팅 + graph-text 블록을 텍스트로 prepend

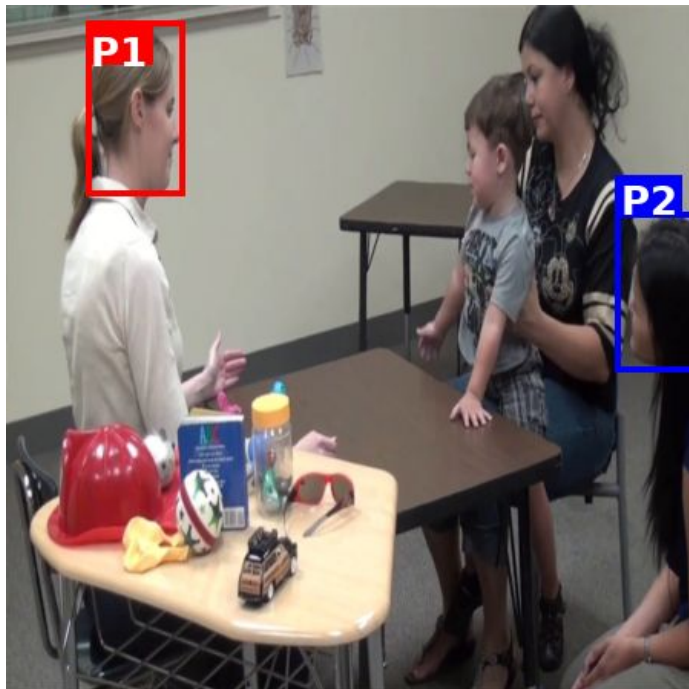
• graph-text (SA의 경우)

- P1 (red) head box [0.12,0.02,0.26,0.28]; P2 (blue) head box [0.89,0.30,1.02,0.53].
- **P1 (red) gaze points right, ~24 deg below horizontal; P2 (blue) gaze points left, ~15 deg above horizontal.**
- **P1's gaze point (0.70,0.23), P2's gaze point (0.23,0.16); distance between the two gaze points: 0.47 (small = same target).**
- **Graph edge scores: shared-attention 0.38, P1->nobody 0.01, P2->nobody 0.01. Per-frame shared-attention scores: 0.21, 0.37, 0.38, 0.39, 0.33.**

• 학습

: LoRA + 답토큰 CE.

실험별 세팅



C. evidence-only graph-text

D. evidence-only graph-text(graph-wrong 4x upsample)

- 입력

: **overlay + graph edge** 판정 없는 **evidence 텍스트**를 질문 앞 prepend.

- 필드

:"[Scene] N people. {i}(red): head box [...], gaze points {direction}, gaze landing at (x,y)." + **task별** 기하 증거만 남김 -

- **LAH**: "{j}(blue): head box [...]. Gaze alignment ({i} toward {j}): +0.XX. Gaze-heatmap overlap with {j}'s head: 0.XX."

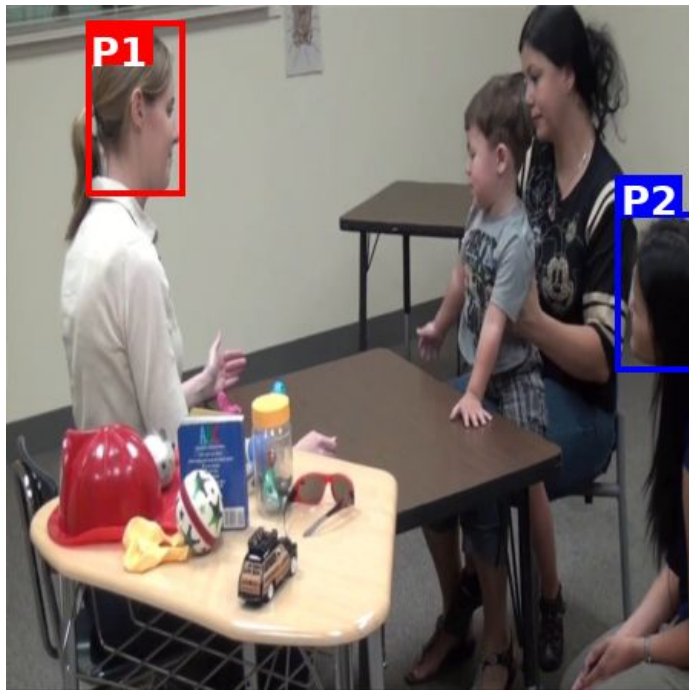
- **LAEO**: 양쪽 gaze field + 양쪽 alignment + 양쪽 overlap.

- **SA**: 양쪽 gaze field + 두 gaze landing point 사이 거리

- 학습

: LoRA + 답토큰 CE.

실험별 세팅



Graph 판정 0.38

Vision-only finetuned VLM no

Vision+Graph finetuned VLM no

E. graph-token (latent soft-token)

• 입력

: overlay + 프롬프트에 <gtok> placeholder 토큰 10개 +질문

• 주입한 10개 토큰 - 각 256-d graph 벡터

1. $v_src[i], v_src[j]$ - i/j를 Looker(emitter) 역할로
2. $v_tgt[i], v_tgt[j]$ - i/j를 Target(receiver) 역할로
3. $edge_pp[i,j], edge_pp[j,i]$ - directed pair 임베딩 (양방향)
4. $edge_null_out[i], edge_null_out[j]$ - 밖 응시
5. $edge_null_in[i], edge_null_in[j]$ - 프레임 안

• 주입 방법

: GraphTokenProjector = Linear(256 → 1024) → GELU → Linear(1024→D_vlm)으로 각 벡터를 hidden 차원으로 투영 → language model에 forward_pre_hook 걸어 <gtok>위치 inputs_embeds를 이 10개 벡터로 덮어씀

• 학습

: LoRA + 답토큰 CE.

VLM Performance

모델	F1_LAH (val/test)	F1_LAEO (val/test)	AP_SA (val/test)
graph (V14.5)	0.8178 / 0.8272	0.7741 / 0.7418	0.7463 / 0.6643
zeroshot-vision	0.462 / 0.450	0.510 / 0.406	0.468 / 0.275
zeroshot-graphtext	0.458 / —	0.593 / —	0.570 / —
finetune-vision	0.7252 / 0.7089	0.7344 / 0.6757	0.7009 / 0.5298
finetune-graphtext	0.8349 / 0.8301	0.7479 / 0.6940	0.7793 / 0.6268
finetune-graphtoken	- / 0.8266	- / 0.6940	- / 0.6268
finetune-evidence-upw	0.7296 / 0.6716	0.7614 / 0.6953	0.7313 / 0.5599

VLM Performance - Soft blend

graph와 VLM의 per-pair 확률 가중 선형결합 $p_{final} = (1-\alpha) \cdot p_{graph} + \alpha \cdot p_{vlm}$ (0.3)

모델	F1_LAH (val/test)	F1_LAEO (val/test)	AP_SA (val/test)
graph (V14.5)	0.8178 / 0.8272	0.7741 / 0.7418	0.7463 / 0.6643
finetune-vision	0.8258 / 0.8279	0.7907 / 0.7608	0.7617 / 0.6667
finetune-graphtext	0.8290 / 0.8287	0.7677 / 0.7368	0.7702 / 0.6676
finetune-graphtoken	- / 0.8272	- / 0.7323	- / 0.6675
finetune-evidence-upw	0.8273 / 0.8277	0.7819 / 0.7422	0.7600 / 0.6492
finetune-multisource	0.8335 /	0.7955 /	0.7747 /

OmniGF: A Dual-Branch Vision-Language Framework for Unified Gaze Following

arxiv 2026.05

Qiaomu Miao¹ Haoyu Wu¹ Jingyi Xu¹ Minh Hoai² Dimitris Samaras¹

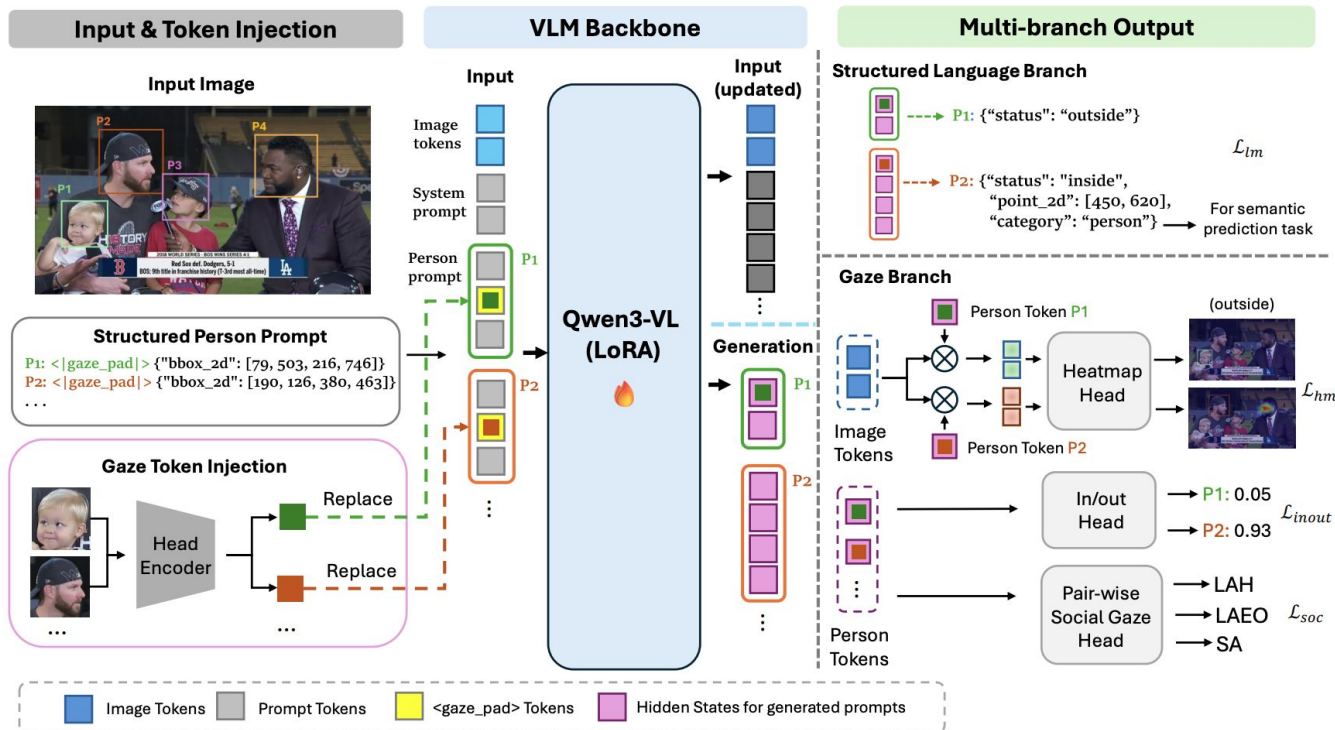
¹Stony Brook University ²The University of Adelaide

VLM은 사람별 gaze reasoning state를 만들고, 실제 좌표/관계 예측은 그 과정에서 생긴 hidden state를 뽑아 dense head가 수행한다.

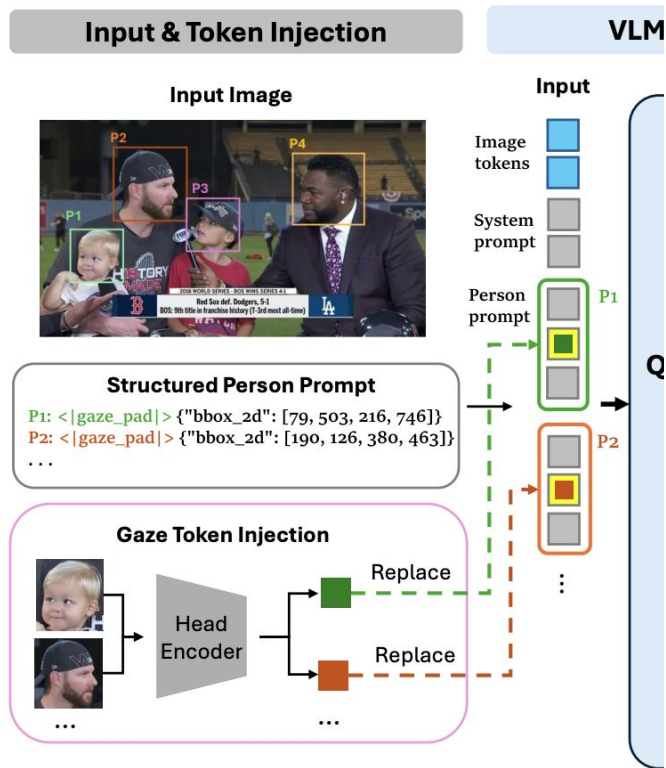
1. **multi-person structured prompt:** 한 이미지 안의 모든 사람을 P1, P2, ...로 한 번에 넣음.
2. **head-conditioned token injection:** 각 사람의 <|gaze_pad|> 토큰 embedding을 head crop encoder output으로 교체.
3. **dual-branch decoding:** language branch는 JSON reasoning을 만들고, gaze branch는 hidden state에서 heatmap / in-out / LAH / LAEO / SA를 예측.

Framework

각 사람마다 $\langle \text{gaze_pad} \rangle$ token을 넣고, 그 token embedding을 crop된 head embedding으로 교체, 이후 VLM이 structured JSON을 생성하고, gaze branch가 생성 과정의 hidden state에서 person token과 image token을 뽑아 heatmap, in/out, social gaze head로 보낸다.



Multi-person prompt + gaze token injection



입력은 image I와 사람 머리 bounding box들

각 사람은 prompt 안에서 이렇게 표현됨

$P_i: <|gaze_pad|> \{ "bbox_2d": [x1, y1, x2, y2] \}$

head crop c_i

↓ Gaze360 pretrained ResNet-18 head encoder

head embedding e_i

↓
replace <|gaze_pad|> token embedding with e_i

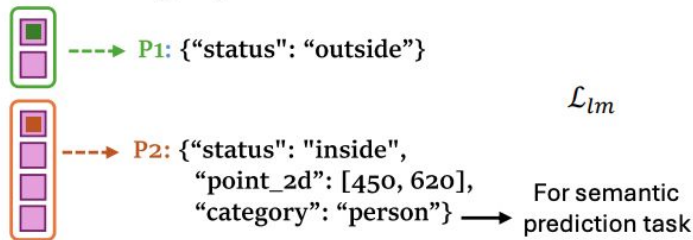
사람별 head appearance/orientation들어난 visual token slot이 됨

이 cue를 VLM token sequence에 직접 주입함.

Structured Language Branch

Multi-branch Output

Structured Language Branch



사람별 JSON 고정해 entire JSON text sequence
standard autoregressive ce loss로 학습

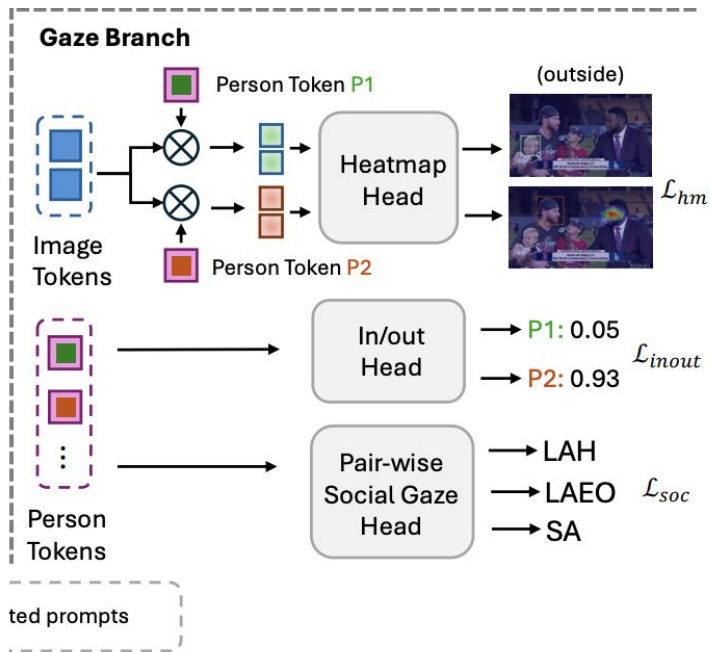
inside일 때 $P_i: \{$
 $\text{"status": "inside",}$
 $\text{"point_2d": [x, y],}$
 $\text{"category": "<object>"} \}$

outside일 때 $P_i: \{$
 $\text{"status": "outside"} \}$

VLM이 생성하는 text coordinate가 spatial precision bottleneck을 가진다고 보고, 최종 localization은 gaze branch의 heatmap에서 가져온다. Language branch의 역할은

1. inside/outside, semantic category 같은 discrete reasoning 생성
2. 사람별 hidden-state anchor 생성 (특히 generated output에서 P_i 의 첫 token hidden state를 사용)

Gaze Branch

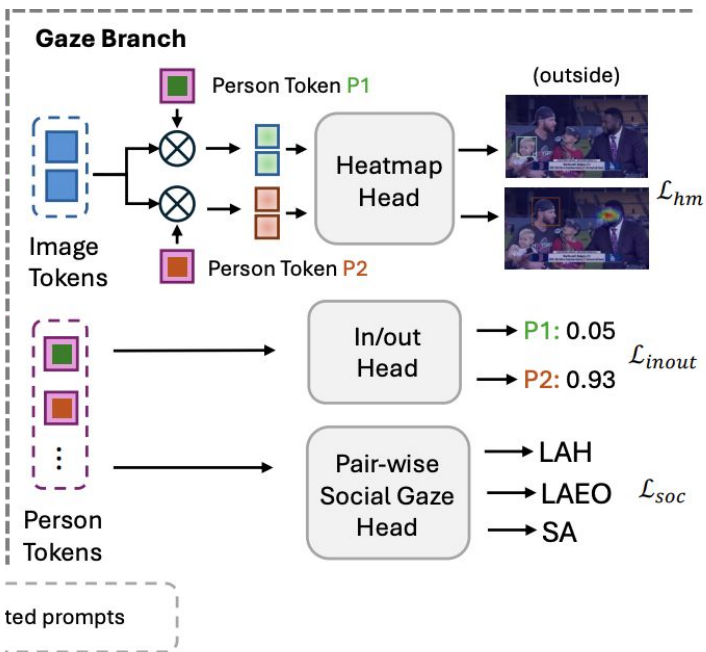


Gaze Branch는 VLM의 마지막 hidden state에서 두 종류의 token을 뽑음.

- 1. person token h_{p_i}**
 - generated " P_i :" 위치의 hidden state
- 2. updated image tokens H'_{vis}**
 - VLM transformer를 통과한 image token들

Gaze Branch

그리고 세 가지 head에 넣음.



1. Heatmap head

$h_{p_i} \times H'_{vis}$ (element-wise multiplication으로 사람 token이 image token을 condition) → **personalized gaze feature** → **deconv decoder** → **heatmap M_i**

GT gaze point 주변에 Gaussian 찍은 target heatmap과 MSE로 학습해 dense한 spatial supervision 받게됨.

2. In/out head

$h_{p_i} \rightarrow \text{MLP} \rightarrow p_{in_i}$

이 사람의 gaze target이 frame 안에 있는지 밖에 있는지 예측, BCE loss 씬.

Gaze Branch

그리고 세 가지 head에 넣음.

3. Pair-wise Social Gaze Head

사람 pair (i,j)에 대해

$[h_{p_i}, h_{p_j}] \rightarrow 3\text{-layer MLP} \rightarrow [LAH, LAEO, SA]$

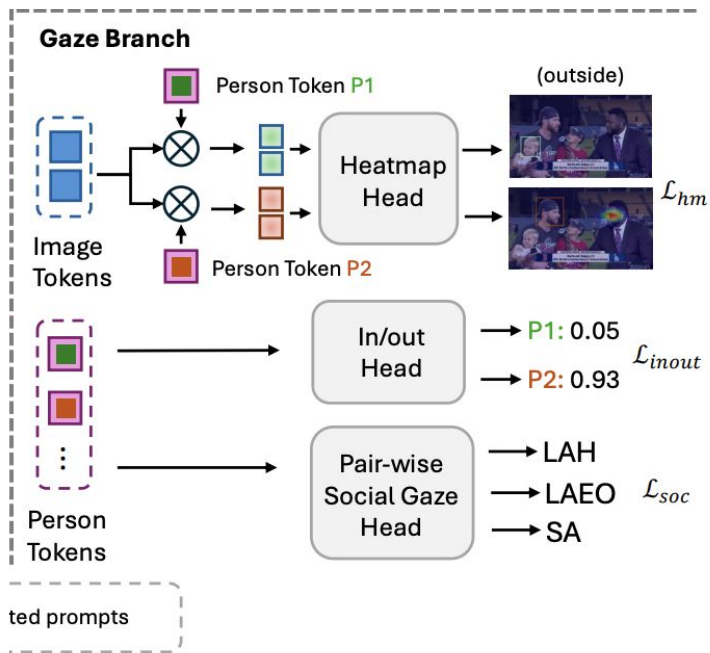
출력은 세 개의 independent logit으로,

LAH : P_i가 어떤 사람을 보고 있는가 / 사람 대상 gaze인가

LAEO : P_i와 P_j가 서로 보고 있는가

SA : P_i와 P_j가 같은 대상을 보고 있는가

이 social head를 BCE로 학습함.



$$L_{total} = \lambda_{lm}(1.0) L_{lm} + \lambda_{hm}(10.0) L_{hm} + \lambda_{inout}(0.1) L_{inout} + \lambda_{soc}(1.0) L_{soc}$$

Results (GazeFollow / VideoAttentionTarget / Childplay)

Table 1: Comparison with state-of-the-art methods on GazeFollow and VideoAttentionTarget. In the column of input modality, I is image, D is depth, P is pose, and E is eyes. Best numbers are marked as bold while second-best are underlined.

Method	Input	Multi-person inference support	GazeFollow		VideoAttentionTarget	
			Avg Dist. ↓	Min Dist. ↓	Dist. ↓	AP ↑
Recasens [11]	I	×	0.190	0.113	–	–
Chong [22]	I	×	0.187	0.112	0.171	0.712
Lian [23]	I	×	0.145	0.081	–	–
Chong [12]	I	×	0.137	0.077	0.134	0.853
Fang [13]	I+D+E	×	0.124	0.067	0.108	0.896
Jin [51]	I	✓	0.126	0.076	0.127	0.882
Bao [24]	I+D+P	×	0.122	–	0.120	0.869
Jin [27]	I+D+P	×	0.118	0.063	0.104	0.895
Gupta [14]	I+D+P	×	0.114	0.056	0.110	0.879
Miao [25]	I+D	×	0.123	0.065	0.109	<u>0.908</u>
Tafasca [28]	I+D	×	0.122	0.062	0.109	0.834
Sharingan [15]	I	✓	0.113	0.057	0.107	0.891
MTGS [38]	I	✓	0.116	0.059	0.105	0.869
Gaze-LLE (ViT-B) [16]	I	×	0.104	0.045	0.107	0.897
Gaze-LLE (ViT-L) [16]	I	×	<u>0.099</u>	<u>0.041</u>	0.103	0.903
GazeVLM [20]	I+D	✓	<u>0.123</u>	<u>0.061</u>	<u>0.102</u>	0.898
OmniGF (proposed)	I	✓	0.091	0.040	0.096	0.923
Human	–	–	0.096	0.040	0.051	0.925

Table 2: Comparison Results on ChildPlay.

Method	Dist. ↓	AP ↑
Gupta [14]	0.113	0.983
Tafasca [28]	0.107	0.986
Sharingan [15]	0.106	0.990
MTGS [38]	0.117	<u>0.993</u>
Gaze-LLE (ViT-B) [16]	0.106	<u>0.994</u>
Gaze-LLE (ViT-L) [16]	<u>0.101</u>	<u>0.994</u>
OmniGF (proposed)	0.090	0.996

Table 3: Ablation Results on GazeFollow.

Components			Metrics	
Language (LoRA)	Spatial Decoding	Token Inject.	Avg ↓	Min ↓
✓			0.340	0.258
			0.114	0.055
✓	✓		0.167	0.103
	✓		0.104	0.047
✓	✓	✓	0.091	0.040

Results (VSGaze)

Method	E2E	Dist. ↓	AP _{IO} ↑	F1 _{LAH} ↑	F1 _{LAEO} ↑	AP _{SA} ↑
Chong _S [12]	×	0.121	0.918	0.778	0.562	0.288
Chong _T [12]	×	0.130	0.956	0.764	0.529	0.331
Gupta [14]	×	0.119	0.929	0.784	0.590	0.335
MTGS [38]	✓	0.107	0.940	0.795	0.590	0.576
OmniGF (proposed)	✓	0.098	0.992	0.814	0.742	0.787

1) Qwen3-VL-4B-Instruct

+ LoRA

+ gaze branch / in-out head

2) GazeFollow에서 localization-only 학습

- L_lm

- L_hm

- L_inout

3) 이 checkpoint를 초기값으로 사용

4) VSGaze에서 social gaze fine-tuning

- L_lm

- L_hm

- L_inout

- L_soc

학습/추론 소요시간

	학습 (teacher-forced)	추론 (autoregressive)
총 per-sample	209 ms (4.78 img/s)	385 ms (2.58 img/s)
지배 비용	LLM fwd+bwd 72%	생성 65%
vision 인코딩	17% (frozen, 캐시가능)	13% (prefill)
heatmap/head	11%	4%

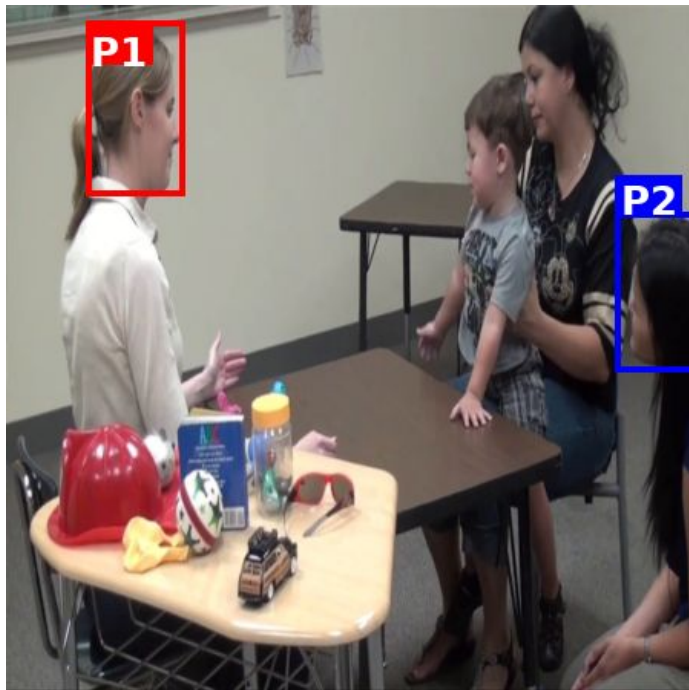
작업	데이터	장수	속도	wall-clock
학습 1 epoch (최적화)	train n≥2	224,851	4.78/s	13.1 h
추론 – test	test	43,581	2.58/s	4.7 h
추론 – val	val	60,862	~2.6/s	~6.5 h

학습/추론 소요시간

Stage-1 Gazefollow 스킵 0.5 epoch 학습 기준	F1_LAH	F1_LAEO	AP_SA
OmniGF 4B (재현)	0.685	0.656	0.475
OmniGF 8B (재현)	0.684	0.675	0.464
OmniGF 논문 (4B)	0.814	0.742	0.787

논문에 정확한 epoch 수 명시되어 있지 않음, 학습 시간 더 최적화해서 여러 epoch 학습시켜봐야 함.

실험별 세팅



Graph 판정 0.38

Vision-only finetuned VLM no

Vision+Graph finetuned VLM no

E. graph-token (latent soft-token)

• 입력

: overlay + 프롬프트에 <gtok> placeholder 토큰 10개 +질문

• 주입한 10개 토큰 - 각 256-d graph 벡터

1. $v_src[i], v_src[j]$ - i/j를 Looker(emitter) 역할로
2. $v_tgt[i], v_tgt[j]$ - i/j를 Target(receiver) 역할로
3. $edge_pp[i,j], edge_pp[j,i]$ - directed pair 임베딩 (양방향)
4. $edge_null_out[i], edge_null_out[j]$ - 밖 응시
5. $edge_null_in[i], edge_null_in[j]$ - 프레임 안

• 주입 방법

: GraphTokenProjector = Linear(256 → 1024) → GELU → Linear(1024→D_vlm)으로 각 벡터를 hidden 차원으로 투영 → language model에 forward_pre_hook 걸어 <gtok>위치 inputs_embeds를 이 10개 벡터로 덮어씀

• 학습

: LoRA + 답토큰 CE.

Experiment Plan

- OmniGF의 Stage 1 Gazefollow pretrain에서 얻을 수 있는 것은 Graph가 대신 준다고 가정.

1. 주입 Graph 토큰에 명시적 기하 정보 추가

→ 현 10토큰에 gaze_point (2), gaze_vec, inout_logit 추가

2. Head-crop token 주입 추가 (OmniGF 방식 적용)

→ 각 person에 ResNet-Gaze360 head embedding을 graph 토큰과 함께 주입

3. Dense heatmap + inout aux supervision (OmniGF 방식 적용)

→ hidden[v_src[i] 위치]로

4. 마지막 yes/no 토큰 대신 social head supervision (OmniGF 방식 적용)

→ hidden [edge_pp[i,j] 위치]로

5. Pair-centric → Multi-person context (OmniGF 방식 적용)

→ 한 번에 pair 하나가 아니라 한 이미지의 모든 Person + relation을 한 forward에

→ Graph가 source → 모든 target hypothesis field 제공할 수 있음.

→ 이 경우 person id overlay 대신 bbox 텍스트 제공